

Assignment: Build a RAG-Based Document Q&A System

Objective

Build a simple **Retrieval-Augmented Generation (RAG)** application that allows a user to upload or provide a document and ask questions about its content.

The objective is to understand how **document processing, chunking, embeddings, vector databases, similarity search, and LLM-based response generation** work together in a RAG pipeline.

Problem Statement

Build a system that can answer questions from a given document without relying only on the model's general knowledge.

Your application should follow this basic flow:

Document → Text Extraction → Chunking → Embeddings → Vector Database → Similarity Search → Retrieved Context → LLM → Answer

Requirements

1. Document Processing

Use at least one document such as:

- PDF
- TXT
- DOCX
- Markdown

Extract the text from the document and prepare it for processing.

2. Text Chunking

Split the extracted text into smaller chunks.

Experiment with:

- Chunk size
- Chunk overlap

Document the values you selected and briefly explain why.

3. Generate Embeddings

Convert each text chunk into an embedding using an embedding model.

The implementation should clearly show:

```
Text Chunk
  ↓
Embedding Model
  ↓
Vector Representation
```

4. Vector Database

Store the generated embeddings along with their corresponding text chunks in a vector database.

You may use any suitable vector database, such as:

- ChromaDB
- FAISS
- Pinecone
- Qdrant
- Weaviate

For a local implementation, **FAISS or ChromaDB** is recommended.

5. Similarity Search

When the user asks a question:

1. Convert the question into an embedding.
2. Search the vector database.
3. Retrieve the most relevant chunks.
4. Pass the retrieved content to the LLM.

Display the retrieved chunks or their metadata so that the retrieval process can be understood.

6. Generate the Final Answer

Use an LLM to generate an answer based on the retrieved context.

The system should instruct the LLM to:

Answer the question using the provided context. If the answer cannot be found in the context, clearly state that the information is not available in the document.

Example

Suppose the document contains information about an organization's policies.

User asks:

"What is the company's work-from-home policy?"

Your system should:

```
graph TD
    Question --> QEmbed[Question Embedding]
    QEmbed --> VectorSearch[Vector Search]
    VectorSearch --> TopRelevant[Top Relevant Chunks]
    TopRelevant --> LLM
    LLM --> FinalAnswer[Final Answer]
```

The answer should be based on the retrieved content from the document.

Mandatory Deliverables

Submit:

1. **Source code**
2. **README.md** containing:
 3. Project overview
 4. Architecture / workflow
 5. Technologies used
 6. Setup instructions
 7. How to run the application
 8. **Sample document** used for testing
9. **Screenshots** showing:
 10. Document ingestion
 11. Embedding/indexing process
 12. Vector database/search
 13. User query
 14. Retrieved context
 15. Final generated answer
16. A short explanation of:
17. What embeddings are

18. Why a vector database is required
19. How similarity search works
20. What chunk size and overlap you selected
21. How RAG differs from simply asking an LLM a question

Bonus Challenges

For additional credit, implement any of the following:

- Support multiple documents
- Add document metadata and metadata filtering
- Display source citations with the answer
- Add a Streamlit or web-based UI
- Allow users to upload PDFs dynamically
- Compare different chunk sizes
- Compare different embedding models
- Display the similarity/relevance score for retrieved chunks
- Implement conversation history
- Add a "I don't know" response when relevant information cannot be retrieved

Evaluation Criteria

Area	Weight
Document processing	10%
Chunking & indexing	15%
Embeddings	15%
Vector database	15%
Similarity search / retrieval	15%
LLM integration	15%
Code quality & README	10%
Demo / explanation	5%

Total: 100%

Expected Outcome

By completing this assignment, you should be able to explain and demonstrate the complete RAG pipeline:

Ingest → Chunk → Embed → Index → Retrieve → Generate

The focus is not just on producing the final answer, but on understanding **what happens internally at each stage of the RAG pipeline**.